

Analysis: Cody's Jams

With the regular and sale prices all jumbled together, how can you determine which prices are which? For the Small dataset, one approach is to consider all of the possibilities. Since there are only up to 8 prices, you can figure out all the different ways to choose half of them. Then, for each of those sets of half of the prices, you can assert that those prices are the sale prices, determine what the set of corresponding regular prices would be, and check the other half of the prices to see if they match that set. Eventually, you'll find the one set for which this works. This method won't work for the Large dataset, though, since it can have up to 2000 prices, and there may be far too many possible ways to choose half of them.

One possible strategy for the Large is to make deductions about the values. For example, a price of 9 cannot possibly be a regular price, because the sale price would be 6.75, which is not an integer. But sometimes the value alone won't give away what sort of price it is. For example, a 12 could be a regular price corresponding to a sale price of 9, or a sale price corresponding to a regular price of 16. There's no way to know without looking at other prices in the list, and you may need to look at other prices to classify *those* prices, and so on. It's possible to write code that does this recursively, but it's a headache. Is there a better way?

Let's call the smallest price in the list $P_{\min}$. A key observation is that $P_{\min}$ *must* be a sale price. (Suppose that $P_{\min}$ were a regular price. Then the list would also have to include an even smaller $3/4 * P_{\min}$ sale version of $P_{\min}$... but then $P_{\min}$ wouldn't be the smallest price in the list anymore!) Then the regular price corresponding to $P_{\min}$ is just $4/3 * P_{\min}$, and you can find that price in the list. You can then remove both $P_{\min}$ and $4/3 * P_{\min}$ from the list, and add $P_{\min}$ to your answer. Once you've done that, you can use the same strategy again: the smallest remaining price must be a sale price, and so on, until you've processed the entire list.

The approach above can be implemented in $O(N^2)$ if you use linear search and removal, $O(N \log N)$ if you use a balanced tree structure to search and remove more efficiently (such as C++'s `set` or Java's `TreeSet`), and $O(N)$ if you use chasing pointers (keep pointers to the last assigned price of each type, and only move those pointers forward when searching).

Analysis: Dance Around The Clock

The Small dataset's limits are small enough that you can create an array of dancers and simulate each turn, and then find dancer **K** and check their neighbors. The circular arrangement of the dancers can make this tricky. You might find the modulo operator useful to avoid special-casing the edges of the array.

In the Large dataset, a 100 million turn simulation with 100 million dancers is unlikely to finish in time, so we need to make some simplifying observations about the dance. At first, the dance might seem intricate, but it follows a very regular pattern. Observe what happens for **D** = 6, **N** = 7. (These lists start from the 12:00 position, and go in clockwise order.)

```
1 2 3 4 5 6
2 1 4 3 6 5
5 4 1 6 3 2
4 5 6 1 2 3
3 6 5 2 1 4
6 3 2 5 4 1
1 2 3 4 5 6
2 1 4 3 6 5
```

Some things to notice:

- The list repeats itself after **D** turns, and will continue to repeat every **D** turns.
- The odd-numbered dancers move clockwise together as a set, one position per turn, always keeping their same relative order. The even-numbered dancers move counterclockwise together as a set, one position per turn, always keeping their same relative order.

Let's follow dancer 3 through those turns. The left neighbors of dancer 3, after 0, 1, 2... turns, are 2, 4, 6, 2, 4, 6, 2, 4. The right neighbors of dancer 3 are 4, 6, 2, 4, 6, 2, 4, 6. Note that on every turn, the numbers of both neighbors go up by 2 (looping around once the maximum value of 6 is reached), and the right neighbor's number is always 2 larger than the left neighbor's number. This makes sense — odd-numbered dancers are always moving one position to the right as the set of even-numbered dancers moves one position past them to the left, so they successively end up between every pair of even-numbered dancers. The cycle of neighbor numbers repeats every **D** / 2 turns.

So, for an odd-numbered dancer numbered **K** out of **D**, the formulas for the left neighbor **L** and the right neighbor **R** after **N** turns are:

$$L = ((K - 2 + 2 * N) \% D) + 1$$
$$R = ((K + 2 * N) \% D) + 1$$

The dancers are numbered from 1 through **N**, but the modulo operator produces values between 0 and **N** - 1. The formulas above effectively transform the dancer numbers into that system and then transform it back. Alternatively, you may find it more convenient in your program to subtract 1 from every dancer's number upon reading the data, and then add 1 just before outputting the answer. Whichever strategy you use, it's worth checking a couple examples with dancers with the smallest and largest possible odd numbers.

We leave the very similar corresponding formulas for even-numbered dancers as an exercise.

The approach we've outlined here — writing a brute force Small solution, inspecting the process or output, and deriving some insights that are helpful for the Large dataset — can be very fruitful in Code Jam, as we've mentioned in our [essay on Small datasets](#).

Analysis: Password Security

To solve this problem, let's start with some general observations about the passwords:

- If any password is only one letter, then the answer is `IMPOSSIBLE` because that password is guaranteed to appear in any permutation of the alphabet. (Incidentally, Google Code Jam does not endorse the use of single-character passwords! An attacker with access to a list of alphabet letters can quickly try them all.)
- If any password has a repeated letter, it can be ignored, since it cannot possibly appear in any permutation of the alphabet.

In Small dataset 1, there's only one password. You might have considered picking a single permutation and hoping that none of the passwords in the test cases happened to be in it. We designed our test cases to thwart that approach: for some letter (say, A), they include a set of fifty test cases BA, CA, ..., ZA, AB, AC, ..., AZ. No matter where the A is in your chosen string, it will be before or after at least one other letter, and is guaranteed to include one of those two-letter passwords, so it will fail at least one case.

One possible approach for Small dataset 1 is to build an alphabet permutation that will definitely not contain each given password:

- If the password is only one letter, return `IMPOSSIBLE`.
- If the password has a repeated letter, it does not affect the answer, and you may return any permutation.
- Otherwise, reverse the password and then tack on whatever remains of the alphabet, and return that.

Another approach is to produce a set of alphabet permutations of which at least one will not contain the password, and then check all of those until you find one that works. For example, you can generate every possible rotation of the alphabet: ABC...XYZ, BC...XYZA, C...XYZAB, and so on. No string of two or more letters can possibly appear in all 26 of those rotations, so one of them must work. Or, you can check the alphabet and the reversed alphabet — again, no string of two or more letters can appear in both of those.

In Small dataset 2, since there are multiple passwords to contend with, the strategies above won't work. Moreover, there are now sets of passwords for which there is no solution, even if none of the passwords is a single letter. One such set is the tricky set of fifty words mentioned above: BA, CA, ..., ZA, AB, AC, ..., AZ. Just as there was no one password that could pass when those fifty words were used as individual test cases, there is no solution when all of them appear in a single test case. Even when there is a solution, you have to find one of the four hundred septillion permutations of the alphabet that works.

One strategy is to try enumerate all of the impossible scenarios. With some thought, it is possible to prove that they always fall into one of three categories:

1. The same letter is required to be at both the beginning and the end. Our AB, AC, ..., AZ, BA, CA, ..., ZA example from before falls into this category. The first 25 bigrams would force A to be at the end (by prohibiting any other letter from appearing after it), and the last 25 bigrams would force A to be at the beginning (by prohibiting any other letter from appearing before it).
2. Two different letters are both required to be at the beginning. For example, consider the set BA, CA, ..., ZA, AB, CB, ..., ZB. The first 25 bigrams would force A to be at the

beginning, and the last 25 bigrams would force B to be at the beginning. But they can't both be there!

3. Two different letters are both required to be at the end. For example: AB, AC, ..., AZ, BA, BC, ..., BZ.

We have a truly marvelous demonstration of this proposition which the margins of this webpage are too narrow to contain¹.

Even if you can't prove to yourself that these are the only three impossible cases, you can always run a simulation on a smaller scenario (with, say, all sets of 10 bigrams from a six-letter alphabet) to increase your confidence.

So, you can explicitly check for these situations, and if the test case doesn't fall into one of those categories, it's guaranteed that at least one answer exists. Now the problem is to find one! You can use various heuristics to try to increase the probability of finding an answer, but you have a perfectly good computer. Why not try a bunch of random permutations until one works?

It is worth noticing that, if the random procedure does find an answer, it will certainly be correct. So, even if you are unconvinced about your impossible case enumeration, you can always try it. If the program finishes, then the output is guaranteed to be correct. If it does not, it could be that the random procedure is too slow in finding a solution, or that you've missed some impossible case.

Most Code Jam problems are written such that random approaches take a prohibitively long time, if they work at all. However, sometimes you can prove to yourself that a random approach is suitable. Consider a case that isn't impossible, but is very close to one of the impossible ones. For example, this case forces ZA (but not BZA) at the end.

AB AC AD AE AF AG AH AI AJ AK AL AM AN AO AP AQ AR AS AT AU AV AW AX AY AZ BA CA
DA EA FA GA HA IA JA KA LA MA NA OA PA QA RA SA TA UA VA WA XA YA BZA

As you can confirm with a simulation, the probability of a random permutation meeting this particular set of conditions turns out to be more than 0.147%. Those might not be great odds for a personal wager, but they're great odds for a computer that can generate and check tens of thousands of alphabet permutations (or more) each second. The odds of finding an answer in 1000 tries are $1 - ((1 - 0.00148)^{1000})$, or a little over 77%. If we swap out the 1000 for 10000 to represent 10000 tries, the odds of finding an answer go up to over 99.9999%. But we also need to consider that there are 100 test cases; if we pretend that none of them are IMPOSSIBLE, that drops the overall chance of succeeding to a mere 99.996%.

So, if you can convince yourself that there can't be cases much worse than the one above, you can just run 1000 times — or, if you want even greater certainty and your program is fast enough, 10000 times — until you find an answer.

Once you're convinced that your random procedure will either find a correct answer (with very high probability) or fail, you can even use a set of 10000 runs with no found answer as "proof" (beyond a reasonable doubt, at least) of impossibility — you don't need to explicitly check for the types of impossible cases mentioned above at all.

Randomized approaches like this are useful outside of programming contest problems. Check out the [Fermat primality test](#), which provides one way of checking whether a number is likely to be prime without actually factoring it. Primality tests like that one are commonly used as part of encryption systems that rely on a fresh supply of large prime numbers.

¹ We do have a full proof, but we leave it as an exercise for mathematically inclined readers to do themselves.

Analysis: Polynesiaglot

In Small dataset 1, all of the cases have only one consonant and only one vowel available. One approach is to designate characters like c for the consonant and v for the vowel, then generate all strings of length L consisting of only c s and/or v s. Then, just find the subset of those strings that follow the "Any consonant in a word must be immediately followed by a vowel" rule, and find the size of that subset.

That subset can be extended to solve Small dataset 2, in which we can have multiple consonants and/or vowels. This time, think of c as meaning "any one of the available consonants" and v as meaning "any one of the available vowels". Then every valid word in the language matches exactly one of these templates, and we just have to figure out how many words match each template. For example, let's consider $C = 2$, $V = 3$, $L = 2$. Then the possible templates are cv and vv . For cv , there are 2 choices for the consonant and 3 choices for the vowel, so there are $2 * 3 = 6$ different valid words match that template. For vv , there are 3 choices for the first vowel and 3 choices for the second vowel, so there are $3 * 3 = 9$ words the match the second template. So the total number of valid words is $6 + 9 = 15$.

What about the Large dataset? For large values of L , there are far too many different templates to generate using the above method.

If you looked carefully at the results for Small dataset 1, you might have noticed a pattern: for $L = 1, 2, 3, 4, 5, 6, 7, \dots$, the results are 1, 2, 3, 5, 8, 13, 21... That's exactly the Fibonacci sequence, in which each term in the sequence equals the sum of the two previous terms. Can we frame this problem in a similarly recursive way?

We will lay out one thorough approach to reaching a recursive formula. Consider any valid word of length X in the language. Imagine removing the first letter. Then the remainder must also be a valid word. This is because removing the first letter can't have created a situation in which a consonant isn't followed by a vowel. (If we'd removed the last letter instead, the leftovers might have ended in a consonant!) So any valid word in the language is either a vowel or a consonant, followed by either some other valid word in the language, or the empty word. (That last part is needed to account for valid words of length 1.)

Let's define $W_c(X)$ as the number of valid words of length X starting with a consonant, $W_v(X)$ as the number of valid words of length X starting with a vowel, and $W(X)$ as the total number of valid words of length X . Then we can write:

$$W_c(X) = C * W_v(X-1)$$

(For any valid word of length X starting with a consonant, the rest of the word can be any valid word of length $X-1$ starting with a vowel. There are C different consonants that we can choose to put in front of any of those words.)

$$W_v(X) = V * W(X-1)$$

(For any valid word of length X starting with a vowel, the rest of the word can be any valid word of length $X-1$. There are V different vowels that we can choose to put in front of any of those words.)

Substitute the second equation into the first to get:

$$W_c(X) = C * V * W(X-2)$$

Finally, $W(X)$ is just $W_v(X)$ plus $W_c(X)$:

$$W(X) = \mathbf{V} * W(X-1) + \mathbf{C} * \mathbf{V} * W(X-2)$$

Since this is a recursive definition, we need some base cases:

$$W(0) = 1$$

(We must treat the empty word as a valid basis for building additional words.)

$$W(1) = \mathbf{V}$$

(There are $\mathbf{V}$ valid words of length 1, since the word can only be one of the $\mathbf{V}$ different vowels, and not one of the consonants.)

You can turn that last equation and the two base cases directly into code. The biggest potential problem is repeating the same sub-calculations over and over again. One way to avoid that is via memoization, a form of [dynamic programming](#): create a structure to hold the results of individual calculations like $W(3)$. Then, before performing any calculation, check the array to see if you already have that result. If you do, just use that; if you don't, then compute the result and store it in the array.

Alternatively, you can avoid storing so many values by starting from $X = 0$ and working up to $X = \mathbf{L}$, instead of working backwards from $X = \mathbf{L}$. Since $W(X)$ only depends on $W(X-1)$ and $W(X-2)$, you can compute $W(2)$ using $W(1)$ and $W(0)$, then compute $W(3)$ using $W(2)$ and $W(1)$ (note that we no longer care about $W(0)$ at this point), and so on. This approach only keeps track of three values at a time, whereas the memoization approach requires $O(\mathbf{L})$ memory. (You can use a similar approach to calculate any Fibonacci number using only two variables.)

What about the requirement to provide the remainder modulo $10^9 + 7$? This is a common situation in programming contests. In languages in which you must work with numbers of fixed sizes, you can take the result of every intermediate operation (addition or multiplication) modulo $10^9 + 7$. You may be tempted to use a language like Python that will handle very large numbers, and perform the modulo operation once at the end, but this is not a good idea in general — large numbers in intermediate calculations can slow algorithms down considerably. In this case, we'd go from $O(\mathbf{L})$ to $O(\mathbf{L}^2)$.

Note that if we have only one vowel and one consonant, as in Small dataset 1, then the last equation reduces to $W(X) = W(X-1) + W(X-2)$. This is exactly the recurrence for the Fibonacci sequence, which explains the results from Small dataset 1.

The solution we have presented requires $O(\mathbf{L})$ operations (i.e., it takes time proportional to $\mathbf{L}$). A better solution can take advantage of the fact that the $\mathbf{L}$ -th term of a sequence defined by a [recurrence relation](#) like the the definition we outlined above can be calculated by using matrix exponentiation with only $O(\log \mathbf{L})$ matrix multiplications. Since in this case the total size of the matrix is bounded, that yields an $O(\log \mathbf{L})$ algorithm that solves the problem. We leave the details as an exercise for the reader.